

CS432: Databases

Structured Query Language (SQL)

Instructor
Yogesh K. Meena

Lecture no.
10

CSE, IIT Gandhinagar
February 11, 2026

Joined Relations

- **Join operations** take two relations and return as a result another relation.
- A join operation is a Cartesian product which requires that tuples in the two relations match (under some condition). It also specifies the attributes that are present in the result of the join
- The join operations are typically used as subquery expressions in the **from** clause

Recap: Natural Join in SQL

- Natural join matches tuples with the same values for all common attributes, and retains only one copy of each common column.
- List the names of instructors along with the course ID of the courses that they taught

Recap: Natural Join in SQL

- Natural join matches tuples with the same values for all common attributes, and retains only one copy of each common column.
- List the names of instructors along with the course ID of the courses that they taught

```
select name, course_id  
from students, takes  
where student.ID = takes.ID;
```

Recap: Natural Join in SQL

- Natural join matches tuples with the same values for all common attributes, and retains only one copy of each common column.
- List the names of instructors along with the course ID of the courses that they taught

```
select name, course_id  
from students, takes  
where student.ID = takes.ID;
```

- Same query in SQL with “natural join” construct

Recap: Natural Join in SQL

- Natural join matches tuples with the same values for all common attributes, and retains only one copy of each common column.
- List the names of instructors along with the course ID of the courses that they taught

```
select name, course_id  
from students, takes  
where student.ID = takes.ID;
```

- Same query in SQL with “natural join” construct

```
select name, course_id  
from student natural join takes;
```

Recap: Natural Join in SQL

- Natural join matches tuples with the same values for all common attributes, and retains only one copy of each common column.
- List the names of instructors along with the course ID of the courses that they taught

```
select name, course_id  
from students, takes  
where student.ID = takes.ID;
```

- Same query in SQL with “natural join” construct

```
select name, course_id  
from student natural join takes;
```

- The from clause can have multiple relations combined using natural join:

```
select  $A_1, A_2, \dots, A_n$   
from  $r_1$  natural join  $r_2$  natural join ... natural join  $r_n$   
where  $P$ ;
```

Student Relation

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>tot_cred</i>
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

Takes Relation

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>grade</i>
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A
12345	CS-347	1	Fall	2017	A
19991	HIS-351	1	Spring	2018	B
23121	FIN-201	1	Spring	2018	C+
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	B+
45678	CS-319	1	Spring	2018	B
54321	CS-101	1	Fall	2017	A-
54321	CS-190	2	Spring	2017	B+
55739	MU-199	1	Spring	2018	A-
76543	CS-101	1	Fall	2017	A
76543	CS-319	2	Spring	2018	A
76653	EE-181	1	Spring	2017	C
98765	CS-101	1	Fall	2017	C-
98765	CS-315	1	Spring	2018	B
98988	BIO-101	1	Summer	2017	A
98988	BIO-301	1	Summer	2018	null

student natural join takes

ID	name	dept_name	tot_cred	course_id	sec_id	semester	year	grade
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2017	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2017	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2017	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2017	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2018	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2017	A
19991	Brandt	History	80	HIS-351	1	Spring	2018	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2018	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2017	B-
45678	Levy	Physics	46	CS-101	1	Fall	2017	F
45678	Levy	Physics	46	CS-101	1	Spring	2018	B+
45678	Levy	Physics	46	CS-319	1	Spring	2018	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2017	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2017	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2018	A-
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2017	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2018	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2017	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2017	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2018	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2017	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2018	null

Issues in Natural Join

- Some tuples in either or both of the relations being joined may be “lost”. e.g., student Snow with ID 70557 has not taken any courses.
- Beware of unrelated attributes with same name which get equated incorrectly. e.g., list the names of students instructors along with the titles of courses that they have taken

Issues in Natural Join

- Some tuples in either or both of the relations being joined may be “lost”. e.g., student Snow with ID 70557 has not taken any courses.
- Beware of unrelated attributes with same name which get equated incorrectly. e.g., list the names of students instructors along with the titles of courses that they have taken

select name, title

from student **natural join** takes **natural join** course

Issues in Natural Join

- Some tuples in either or both of the relations being joined may be “lost”. e.g., student Snow with ID 70557 has not taken any courses.
- Beware of unrelated attributes with same name which get equated incorrectly. e.g., list the names of students instructors along with the titles of courses that they have taken

select name, title

from student **natural join** takes **natural join** course

This query omits all (student name, course title) pairs where the student takes a course in a department other than the student's own department.

- Correct query:

Issues in Natural Join

- Some tuples in either or both of the relations being joined may be “lost”. e.g., student Snow with ID 70557 has not taken any courses.
- Beware of unrelated attributes with same name which get equated incorrectly. e.g., list the names of students instructors along with the titles of courses that they have taken

```
select name, title
```

```
from student natural join takes natural join course
```

This query omits all (student name, course title) pairs where the student takes a course in a department other than the student's own department.

- Correct query:

```
select name, title
```

```
from student natural join takes, course
```

```
where takes.course_id = course.course_id;
```

Natural Join with Using Clause

- To avoid the danger of equating attributes erroneously, we can use the “using” construct that allows us to specify exactly which columns should be equated.

- **Example**

```
select name, title
from (student natural join takes) join course using (course_id)
```

Join Condition

- The **on** condition allows a general predicate over the relations being joined
- This predicate is written like a **where** clause predicate except for the use of the keyword **on**
- **Example**

select*

from student join takes on student_ID = takes_ID

Join Condition

- The **on** condition allows a general predicate over the relations being joined
- This predicate is written like a **where** clause predicate except for the use of the keyword **on**
- **Example**

```
select*
```

```
from student join takes on student_ID = takes_ID
```

```
select*
```

```
from student, takes
```

```
where student_ID = takes_ID
```

Join Condition

- The **on** condition allows a general predicate over the relations being joined
- This predicate is written like a **where** clause predicate except for the use of the keyword **on**

- **Example**

```
select*
```

```
from student join takes on student_ID = takes_ID
```

```
select*
```

```
from student, takes
```

```
where student_ID = takes_ID
```

- The on condition specifies that a tuple from student matches a tuple from takes if their ID values are equal.
- ID attribute will be listed twice (see student natural join takes relation table)

Join Condition (Cont.)

- **Example:** Query that displays the ID values only once



Join Condition (Cont.)

- **Example:** Query that displays the ID values only once
`select student_ID as ID, name, dept_name, tot_cred, course_id,
sec_id, semester, year, grade
from student join takes on student_ID = takes_ID`
- The **on** condition allows for more expressive and flexible join conditions compared to a **natural join**.
- Although the **on** condition can sometimes be replaced by the **where** clause, **why it is preferred?**

Join Condition (Cont.)

- **Example:** Query that displays the ID values only once
`select student_ID as ID, name, dept_name, tot_cred, course_id,
sec_id, semester, year, grade
from student join takes on student_ID = takes_ID`
- The **on** condition allows for more expressive and flexible join conditions compared to a **natural join**.
- Although the **on** condition can sometimes be replaced by the **where** clause, **why it is preferred?**
 - Behavioral Differences in Outer Joins:** The **on** condition preserves unmatched rows in **outer joins**, whereas the **where** clause may filter them out.
 - Improved Readability:** Placing the join condition in the **on** clause and filtering conditions in the **where** clause makes SQL queries clearer and easier to understand.

Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- Uses *null* values.
- **Example:** Display a list of all students, displaying their ID, and name, dept_name, and tot_cred, along with the courses that they have taken.

Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- Uses *null* values.
- **Example:** Display a list of all students, displaying their ID, and name, dept_name, and tot_cred, along with the courses that they have taken.

```
select*  
from student natural join takes;
```

Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- Uses *null* values.
- **Example:** Display a list of all students, displaying their ID, and name, dept_name, and tot_cred, along with the courses that they have taken.

```
select*  
from student natural join takes;
```

— Problem with the this query?

Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- Uses *null* values.
- **Example:** Display a list of all students, displaying their ID, and name, dept_name, and tot_cred, along with the courses that they have taken.

select*

from student natural join takes;

— **Problem with the this query?**

— Outer joins **preserve** those tuples that would be lost in a join, by creating tuples in the result containing null values.

Three forms of outer join

- The left outer join
- The right outer join
- The full outer join

Previous ones are termed as **inner join operations**.



How outer joins operates

Left outer-joins

- Compute the result of the inner join as before.
- For every tuple t in the left-hand-side relation that does not match any tuple in the right-hand-side relation in the inner join, add a tuple r to the result of the join.

Left outer-joins: example

```
select*  
from student natural left outer join takes;
```



Left outer-joins: example

`select*`
`from student natural left outer join takes;`

ID	name	dept_name	tot_cred	course_id	sec_id	semester	year	grade
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2009	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2009	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2009	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2009	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2010	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2009	A
19991	Brandt	History	80	HIS-351	1	Spring	2010	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2010	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2009	B-
45678	Levy	Physics	46	CS-101	1	Fall	2009	F
45678	Levy	Physics	46	CS-101	1	Spring	2010	B+
45678	Levy	Physics	46	CS-319	1	Spring	2010	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2009	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2009	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2010	A-
70557	Snow	Physics	0	null	null	null	null	null
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2009	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2010	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2009	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2009	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2010	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2009	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2010	null

Right outer-joins: example

`select*`
`from takes natural right outer join student;`



Full outer join

- Display a list of all students in the Comp. Sci. department, along with the course sections, if any, that they have taken in Spring 2009; all course sections from Spring 2009 must be displayed, even if no student from the Comp. Sci. department has taken the course section.

```
select*  
from (select*  
      from student  
      where dept_name = 'Comp.Sci'  
      natural full outer join  
      select*  
      from takes  
      where semester = 'Spring' and year= 2009;
```

'on' and 'where' with outer joins

`select*`
`from student left outer join takes on student.ID = takes.ID;`

'on' and 'where' with outer joins

select*
from student **left outer join** takes **on** student.ID = takes.ID;

select*
from student **left outer join** takes **on** true
where student.ID = takes.ID;

Join Types and Conditions

- **Join operations** take two relations and return as a result another relation.
- These additional operations are typically used as subquery expressions in the **from** clause
- **Join condition:** defines which tuples in the two relations match.
- **Join type:** defines how tuples in each relation that do not match any tuple in the other relation (based on the join condition) are treated.

<i>Join types</i>
inner join
left outer join
right outer join
full outer join

<i>Join conditions</i>
natural
on <predicate>
using ($A_1, A_2, \dots, A_n$)

Views

- Relations in the collection are the actual relations stored in the database.
- Not desirable for all users to see the entire logical model.
- Certain data be hidden from users.
- Example: consider a person who needs to know an instructors name and department, but not the salary. This person should see a relation described, in SQL, by

```
select ID, name, dept_name  
from instructor
```

View Definition

- A view is defined using the **create view** statement which has the form
— **create view** *v* **as** <query expression>;
- Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.
- **Note:** View definition is not the same as creating a new relation by evaluating the query expression. Rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.

Transactions

A transaction consists of a sequence of query and/or update statements.

- Two ending statements:
 - Commit work
 - Rollback work

Commit work

- Commits the current transaction
- Updates performed by the transaction become permanent in the database
- After commit, a new transaction can be automatically started

Saving changes to a document that is being edited

Rollback work

- To roll back the current transaction
- Undoes all the updates performed by the SQL statements in the transaction
- The database state is restored to what it was before the first statement of transaction

Quit edit session without saving changes

Few rules

- Once a transaction has executed 'commit work', its effects can no longer be undone by 'rollback work'
- In case of a power outage, or a system crash, a transaction's effects will be rolled back if it has not yet executed 'commit work'
- In case of power outage, the 'rollback work' occurs when the system restarts

Some examples

- Transfer money from one bank account to another
 - debit money from one account
 - credit money to another account
- Updating credits in the student relation
 - Add entry in the 'takes' relation
 - update 'tot_cred' attribute in 'student' relation

Either all the effects of the transaction are reflected in the database, or none are

Transaction in real implementations

- Each SQL statement is a transaction, and gets committed just after execution
- Transaction consisting of multiple SQL statements needs to be executed, then default settings must be turned off.
- In SQL:1999, **begin atomic . . . end**

More discussion in Chapter 14 of Text Book

Integrity Constraints

Constraints on a Single Relation

- Not null
- Unique
- Check (<predicate>)
 - Check(P) specifies a predicate P that must be satisfied by every tuple in a relation.

```
create table section
  (course_id      varchar (8),
   sec_id         varchar (8),
   semester       varchar (6),
   year           numeric (4,0),
   building       varchar (15),
   room_number    varchar (7),
   time_slot_id   varchar (4),
   primary key (course_id, sec_id, semester, year),
   check (semester in ('Fall', 'Winter', 'Spring', 'Summer')));
```

Constraints on a Multiple Relation

Referential Integrity

- For each course tuple, the department name specified in the tuple must exist in the department relation

```
create table course
(course_id      varchar (8),
 title         varchar (50),
 dept_name     varchar (20),
 credits       numeric (2,0) check (credits > 0),
primary key (course_id),
foreign key (dept_name) references department)
```

Authorization

Several forms of authorizations

- Authorization to read data.
- Authorization to insert new data.
- Authorization to update data.
- Authorization to delete data.

Each of these types of authorizations is called a **privilege**.

select, insert, update, and delete

Granting of Privileges

A user who creates a new relation is given all privileges on that relation automatically.

- A generic template
 - **grant** < privileges list>
 - **on** < relation name of view name>
 - **to** <user/role list>;
- Select privilege
 - **grant select on** department **to** Adam, Bak;

Acknowledgments/Contributions

- Some of the images utilized in these slides are subject to copyright - Abraham Silberschatz, Henry Korth, and S. Sudarshan. Database System Concepts. 6th Edition, McGraw-Hill Education
- Contributor 2024-25, 2025-26 - Yogesh K. Meena, IIT Gandhinagar